

RAPPORT DE PROJET

Phase 2 — Authentication & Base de données

Projet : Secure API — Cybersécurité

1. Introduction

La Phase 2 du projet Secure API constitue le coeur fonctionnel de l'application : la mise en place du système d'authentification et de la base de données. Cette phase implémente deux mécanismes de sécurité fondamentaux en cybersécurité : le chiffrement des mots de passe avec bcrypt et l'authentification par tokens JWT.

À l'issue de cette phase, l'API est capable d'inscrire des utilisateurs (en chiffrant leur mot de passe), de les authentifier (en vérifiant leur mot de passe et en générant un token), et de stocker toutes les données dans une base de données SQLite.

1.1 Objectifs de la Phase 2

- Créer et initialiser une base de données SQLite
- Définir des types TypeScript pour tous les objets métier
- Implémenter l'inscription avec chiffrement bcrypt du mot de passe
- Implémenter la connexion avec génération de token JWT
- Tester l'API avec Postman
- Appliquer les bonnes pratiques de sécurité (messages d'erreur génériques, validation des inputs)

2. Définitions des concepts clés

2.1 La base de données

Base de données : Système organisé pour stocker, gérer et récupérer des données. Dans notre projet, on utilise SQLite — une base de données légère qui stocke toutes les données dans un seul fichier (database.sqlite). Contrairement à MySQL ou PostgreSQL qui nécessitent un serveur dédié, SQLite fonctionne directement dans l'application.

SQL (Structured Query Language) : Langage standard pour interagir avec une base de données relationnelle. Il permet de créer des tables, d'insérer des données, de les lire, de les modifier et de les supprimer. Exemple : `SELECT * FROM users WHERE email = 'test@gmail.com'`

Table : Structure d'une base de données relationnelle, similaire à un tableau Excel. Chaque ligne est un enregistrement (un utilisateur), chaque colonne est un champ (id, username, email, password...).

Requête préparée (Prepared Statement) : Méthode sécurisée pour exécuter des requêtes SQL. Au lieu de concaténer directement les données dans la requête SQL (ce qui est

dangereux), on utilise des paramètres (?) que la bibliothèque remplace de manière sécurisée. C'est la protection principale contre les injections SQL.

2.2 Le chiffrement des mots de passe

Chiffrement : Transformation d'une donnée en une forme illisible pour quiconque ne possède pas la clé de déchiffrement. Dans le contexte des mots de passe, on parle plus précisément de hachage (hash) — une transformation à sens unique.

Hash (Hachage) : Transformation irréversible d'une donnée en une empreinte de taille fixe. Contrairement au chiffrement classique, un hash ne peut pas être déchiffré. Pour vérifier un mot de passe, on hache le mot de passe saisi et on compare le hash obtenu avec celui stocké en base.

bcrypt : Algorithme de hachage spécialement conçu pour les mots de passe. Ses avantages par rapport à des algorithmes comme MD5 ou SHA1 : il est intentionnellement lent (ce qui rend les attaques par force brute très difficiles), il intègre automatiquement un salt (valeur aléatoire ajoutée avant le hachage pour rendre chaque hash unique même pour des mots de passe identiques), et son coût (salt factor) est paramétrable.

Salt Factor : Paramètre de bcrypt qui détermine le niveau de sécurité et la lenteur du hachage. Dans notre projet, on utilise 12, ce qui signifie $2^{12} = 4096$ itérations. Plus ce nombre est élevé, plus le hash est sécurisé mais lent à calculer. Un salt factor de 12 est considéré comme robuste en 2026.

Rainbow Table : Base de données pré-calculée associant des mots de passe courants à leurs hashes. Sans salt, un attaquant peut retrouver un mot de passe en quelques secondes en comparant son hash avec cette table. Avec bcrypt et son salt automatique, cette attaque est rendue impossible car chaque hash est unique.

2.3 L'authentification JWT

Authentification : Processus de vérification de l'identité d'un utilisateur. Ne pas confondre avec l'autorisation (vérification de ce qu'un utilisateur a le droit de faire). L'authentification répond à la question 'Qui es-tu ?' et l'autorisation à 'As-tu le droit de faire ça ?'

JWT (JSON Web Token) : Standard ouvert (RFC 7519) pour la transmission d'informations de manière sécurisée entre parties sous forme de token. Un token JWT est composé de 3 parties encodées en Base64 séparées par des points : le header (algorithme), le payload (données) et la signature.

Token : Jeton numérique signé qui prouve l'identité d'un utilisateur. Une fois connecté, l'utilisateur reçoit un token qu'il doit envoyer dans chaque requête suivante. Le serveur vérifie la signature du token pour confirmer son authenticité sans avoir à interroger la base de données.

Authentication Stateless : Méthode d'authentification où le serveur ne stocke aucune information de session. Toutes les informations nécessaires sont contenues dans le token lui-même. C'est l'opposé de l'authentification par session (stateful) où le serveur doit mémoriser l'état de chaque connexion.

Expiration du token (expiresIn) : Durée de validité d'un token JWT. Dans notre projet, les tokens expirent après 24h. Après expiration, l'utilisateur doit se reconnecter pour obtenir un nouveau token. Cela limite l'impact d'un token volé : l'attaquant ne peut l'utiliser que pendant 24h maximum.

2.4 Architecture et patterns

Interface TypeScript : Contrat qui définit la structure exacte d'un objet. Par exemple, l'interface User spécifie que tout objet utilisateur doit avoir un id (number), un username (string), un email (string), etc. TypeScript vérifie que tous les objets respectent bien ce contrat.

Controller : Composant qui contient la logique métier d'une fonctionnalité. Dans notre projet, authController.ts contient les fonctions register() (inscription) et login() (connexion). Le controller reçoit la requête, applique la logique, et construit la réponse.

Route : Association entre une URL et une fonction. Par exemple, router.post('/register', register) signifie : quand une requête POST arrive sur /register, exécute la fonction register(). Les routes servent d'aiguillage des requêtes vers les bons controllers.

Middleware : Fonction intermédiaire qui s'exécute entre la réception d'une requête et l'envoi de la réponse. Les middlewares servent à effectuer des traitements communs : vérifier un token JWT, valider les données, gérer les logs, etc.

3. Étapes réalisées

3.1 Création de la base de données

Le fichier src/database/database.ts établit la connexion avec SQLite et initialise la table des utilisateurs au démarrage du serveur. La table users comporte les colonnes suivantes :

Colonne	Type	Description	Contrainte
---------	------	-------------	------------

id	INTEGER	Identifiant unique	PRIMARY KEY AUTOINCREMENT
username	TEXT	Nom d'utilisateur	UNIQUE NOT NULL
email	TEXT	Adresse email	UNIQUE NOT NULL
password	TEXT	Mot de passe hashé	NOT NULL
role	TEXT	Rôle (user/admin)	DEFAULT 'user'
created_at	DATETIME	Date de création	DEFAULT CURRENT_TIMESTAMP

L'instruction `CREATE TABLE IF NOT EXISTS` garantit que la table n'est créée que si elle n'existe pas déjà ce qui permet de redémarrer le serveur sans perte de données.

3.2 Définition des types TypeScript

Le fichier `src/types/index.ts` définit 4 interfaces TypeScript :

- `User` : structure complète d'un utilisateur en base de données
- `RegisterInput` : données attendues lors d'une inscription (username, email, password)
- `LoginInput` : données attendues lors d'une connexion (email, password)
- `JwtPayload` : contenu du token JWT (userId, username, role)

Ces interfaces servent de contrats dans tout le code. Si une fonction reçoit un objet qui ne respecte pas l'interface attendue, TypeScript signale une erreur immédiatement, avant même l'exécution du code.

3.3 Implémentation de l'inscription

La fonction `register()` dans `authController.ts` suit ce processus :

Étape	Action	Raison de sécurité
1	Vérifier la présence de tous les champs	Éviter les erreurs et les insertions incomplètes
2	Vérifier la longueur du mot de passe (min. 6 caractères)	Empêcher les mots de passe trop faibles
3	Hacher le mot de passe avec bcrypt (salt factor 12)	Ne jamais stocker un mot de passe en clair
4	Insérer l'utilisateur avec une requête préparée	Protection contre les injections SQL

5	Retourner un code 201 avec l'ID créé	Confirmer la création sans exposer le mot de passe
---	--------------------------------------	--

En cas d'erreur UNIQUE constraint failed (email ou username déjà pris), un message d'erreur explicite est retourné avec le code 409 Conflict.

3.4 Implémentation de la connexion

La fonction login() dans authController.ts suit ce processus :

Étape	Action	Raison de sécurité
1	Vérifier la présence des champs	Validation basique des inputs
2	Rechercher l'utilisateur par email	Requête préparée pour éviter les injections SQL
3	Vérifier le mot de passe avec bcrypt.compare()	Comparaison sécurisée sans déchiffrement
4	Retourner 'Email ou mot de passe incorrect' dans les deux cas d'échec	Ne pas révéler si l'email existe
5	Générer un token JWT signé avec expiration 24h	Authentification stateless sécurisée
6	Retourner le token et les infos utilisateur (sans le mot de passe)	Ne jamais exposer le hash du mot de passe

3.5 Bonne pratique : Message d'erreur générique

Un point de sécurité important : lors de la connexion, qu'il s'agisse d'un email inexistant ou d'un mot de passe incorrect, le serveur retourne TOUJOURS le même message :

```
"Email ou mot de passe incorrect"
```

Si le serveur renvoyait des messages différents selon le cas (Email introuvable vs Mot de passe incorrect), un attaquant pourrait énumérer les emails existants dans la base de données — ce qui constitue une fuite d'information. Ce type d'attaque s'appelle une User Enumeration Attack.

3.6 Tests avec Postman

Postman est l'outil standard en entreprise pour tester les APIs REST. Il permet d'envoyer n'importe quel type de requête HTTP avec un corps JSON personnalisé — ce qu'un navigateur web classique ne peut pas faire.

Tests réalisés avec succès :

Test	Requête	Résultat obtenu
Inscription	POST /api/auth/register avec username, email, password	201 Created — userId: 1
Connexion	POST /api/auth/login avec email, password	200 OK — token JWT généré

Le token JWT reçu lors de la connexion ressemble à :

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiaW9F9.2kfVk_0KQs...
```

Ce token en 3 parties contient : l'algorithme de signature (HS256), les données de l'utilisateur (userId, username, role), et la signature cryptographique qui garantit que le token n'a pas été falsifié.

4. Sécurité mise en place

4.1 Récapitulatif des mesures de sécurité

Mesure	Technologie	Protection contre
Hachage des mots de passe	bcrypt (salt factor 12)	Vol de base de données, Rainbow Tables
Tokens d'authentification	JWT (expiration 24h)	Usurpation d'identité, Sessions infinies
Requêtes préparées	better-sqlite3 prepare()	Injections SQL
Validation des inputs	Vérifications manuelles	Données malformées, attaques basiques
Messages d'erreur génériques	Même message pour email/MDP invalide	User Enumeration Attack
Variables d'environnement	.env + .gitignore	Fuite de clés secrètes sur GitHub

5. Bilan

5.1 Ce qui a été réalisé

Fonctionnalité	Statut
Base de données SQLite initialisée	Réalisé
Table users avec tous les champs	Réalisé
Interfaces TypeScript définies	Réalisé
Endpoint POST /api/auth/register	Réalisé
Endpoint POST /api/auth/login	Réalisé
Chiffrement bcrypt (salt factor 12)	Réalisé
Génération de token JWT (24h)	Réalisé
Tests Postman réussis	Réalisé
Commit GitHub documenté	Réalisé

5.2 Prochaines étapes — Phase 3

- Middleware JWT pour protéger des routes
- Route GET /users accessible uniquement aux utilisateurs authentifiés
- Introduction de failles volontaires (SQL Injection, XSS, mauvaise gestion JWT)
- Démonstration et correction de chaque faille
- Tests avec Burp Suite et OWASP ZAP